PROJECT PROPOSAL AND DESIGN DOCUMENTATION

ORVO

Discover. Prepare. Achieve.

A Proposed Unified Platform for Student Career-Opportunity Discovery and Preparation

Team RUDRAX

Dheeraj Mandal
Jatin Lawaniya
Aakash Pal
Abhishek Tyagi

Team leader: none designated

College: [College name: pending confirmation]
Department: [Department: pending confirmation]
Course: [Course: pending confirmation]
Academic year: [Academic year: pending confirmation]
Repository: [Repository URL: pending confirmation]

Status: pre-development proposal. No part of ORVO has been implemented.

Declaration (Draft for Approval)

We, the members of Team RUDRAX, propose this document as the project proposal and design documentation for ORVO. The content describes planned functionality and design decisions only. We declare that no implementation, testing or deployment is claimed in this document, that sources used are cited in the References, and that all statements remain subject to approval by the team and the institution.

Signatures and dates: pending team approval.

Acknowledgement (Draft)

The team expresses its gratitude to its faculty members and institution for guidance and the opportunity to undertake this project. *Names of mentors and institution: pending confirmation.*

Table of Contents

Abstract **4**
1 Introduction and Background **4**
2 Problem Statement **4**
3 Motivation and Objectives **4**
3.1 Motivation 4
3.2 Objectives 4
4 Existing Approaches and Proposed Solution **5**
4.1 Existing approaches 5
4.2 Proposed solution 5
4.3 Proposed modules 5
5 Scope, Boundaries and Limitations **5**
6 Requirements Specification **6**
6.1 Functional requirements 6
6.2 Non-functional requirements 6
6.3 Business rules 7
7 System Architecture and Design **7**
8 Workflow and Use Cases **9**
9 Database Design **10**
10 Technology Stack and Rationale **12**
11 Security and Privacy Considerations **12**
12 Feasibility Analysis **13**
13 Proposed Test Plan **13**
14 Development Plan and Milestones **14**
15 Proposed Team Responsibilities **15**
16 Expected Outcomes **15**
17 Risks, Assumptions and Constraints **16**
18 Future Scope **16**
19 Conclusion **16**
References **16**
Appendix A: Illustrative API Endpoints **17**
Appendix B: Unresolved Decisions and Placeholders **18**

Abstract

ORVO is a proposed web platform that unifies two activities students currently perform on separate websites: discovering career and skill-development opportunities, and preparing for them. The platform would list hackathons, internships, jobs and freelancing opportunities with eligibility details, deadlines and official application links, and would connect each opportunity to relevant tutorials, practice questions, project guides and preparation material. Students could save opportunities and track deadlines and preparation progress, while administrators would curate and maintain content accuracy.

The proposed implementation uses HTML, CSS and JavaScript for the frontend, Java with Spring Boot for the backend and MySQL for storage, with Git and GitHub for collaboration. This report defines the problem, objectives, scope, requirements, architecture, data model, security considerations, test plan and development plan, and identifies a realistic Minimum Viable Product (MVP) for a four-member student team. The project is at the proposal stage; no feature has been implemented and no test has been run.

1 Introduction and Background

Students are encouraged to take part in hackathons, apply for internships and jobs, explore freelancing and build skills through projects. Information about such opportunities is spread across company career pages, specialised portals, social media and college notices. Preparation material such as tutorials, practice problems and project ideas is spread across a different set of websites. A student therefore repeats a cycle: find an opportunity, work out what it requires, then search elsewhere for ways to prepare, often close to the deadline.

ORVO (tagline: *Discover. Prepare. Achieve.*) is proposed to shorten this cycle by presenting the opportunity and the means of preparing for it in one organised platform. The long-term vision is to grow ORVO into a broader student career-growth platform; this proposal, however, concentrates on a feasible first version.

2 Problem Statement

Students discover opportunities on one platform but must search separately for tutorials, practice questions, project guidance, preparation material and deadline information. No single organised place connects an opportunity with the preparation needed for it, which costs time, causes missed deadlines and makes requirements harder to understand.

3 Motivation and Objectives

3.1 Motivation

The main motivations are saving students' time, reducing missed deadlines, making eligibility and requirements easier to understand, and treating discovery and preparation as one continuous journey.

3.2 Objectives

1. Provide a single place to browse, search and filter opportunities.
2. Show eligibility, deadlines and official application links for each opportunity.
3. Link each opportunity to relevant preparation resources.

4. Allow students to save opportunities and track deadlines and preparation progress.
5. Provide administrators with tools to keep information accurate and current.
6. Establish a maintainable architecture that can later support broader career-growth features.

4 Existing Approaches and Proposed Solution

4.1 Existing approaches

Table 1: Existing approaches and their limitations relevant to ORVO (general characterisation; see References)

Approach (examples)	Strength	Limitation relevant to ORVO
Opportunity portals and job boards (e.g. Devpost for hackathons, Internshala for internships, LinkedIn for jobs)	Large listings in a focused area	Usually concentrate on listing and applying; preparation material is outside their main purpose
Preparation platforms (e.g. LeetCode for coding practice, Coursera for courses)	Structured learning and practice	Not organised around specific opportunities and their deadlines
Social media, college notices, messaging groups	Fast sharing	Unstructured, hard to search, no deadline tracking

ORVO does not claim to replace these platforms. Its proposed contribution is the connection between discovery and preparation while preserving links to official sources.

4.2 Proposed solution

ORVO is proposed as a web application with a student-facing interface and an administrator interface, supported by a REST API and a relational database. Administrators curate opportunities and resources, classify them by category and link them to each other. Students search, filter, save and track. Applications are completed on the official external websites; ORVO does not process applications.

4.3 Proposed modules

Table 2: Proposed modules and responsibilities

Module	Responsibility
Opportunity Discovery	Listing, search, filter, details
Preparation Resources	Resource listing, resource-to-opportunity links
User Accounts	Registration, login, roles
Saved Items and Tracking	Saved opportunities, deadline view, progress
Administration	CRUD for opportunities, resources, categories; publication status

5 Scope, Boundaries and Limitations

In scope (proposed MVP): browsing, search and filtering; opportunity details; resource access; registration and login; saved opportunities with a deadline view; preparation-progress tracking; administrator content management.

Out of scope for the MVP: live scraping or automated aggregation from other sites; AI recommendations; team matching; personalised preparation plans; e-mail or push notifications; third-party sign-in; in-platform applications; native mobile apps; payments.

Limitations: content depends on manual entry and can become outdated; ORVO is a reference tool and students must confirm details on the official site; search in the MVP is limited to database queries.

6 Requirements Specification

The requirements below are proposed and pending team approval (Priority: M = Must, S = Should). The complete specification is in the repository document *02 Software Requirements (SRS)*.

6.1 Functional requirements

Table 3: Proposed functional requirements

ID	Requirement	Pri
FR-01	The system shall list published opportunities.	M
FR-02	The system shall allow keyword search over opportunity title, description and required skills.	M
FR-03	The system shall allow filtering by category/type, required skills, eligibility text, and deadline, where the data exists for the listing.	M
FR-04	The system shall show opportunity details: description, category, eligibility, skills, deadline, official application link.	M
FR-05	The system shall show the preparation resources linked to an opportunity.	M
FR-06	The system shall list and search preparation resources (tutorials, practice questions, project guides, learning material) independently of opportunities.	S
FR-07	The system shall let a visitor register and log in, and let a user log out.	M
FR-08	A logged-in student shall be able to save and unsave an opportunity.	M
FR-09	A student shall be able to view saved opportunities sorted by deadline, with passed deadlines clearly marked.	M
FR-10	A student shall be able to set a preparation status (not started, in progress, completed) for a resource.	S
FR-11	An administrator shall be able to create an opportunity with required fields.	M
FR-12	An administrator shall be able to edit, archive or delete an opportunity.	M
FR-13	An administrator shall be able to create, edit and delete resources.	M
FR-14	An administrator shall be able to link and unlink resources to/from opportunities.	M
FR-15	Each opportunity shall have a status (draft, published, archived); only published items are visible to students.	M
FR-16	The system shall mark opportunities whose deadline has passed as expired.	M
FR-17	An administrator shall be able to manage categories.	S

6.2 Non-functional requirements

Numeric targets are provisional and have no measured basis yet.

Table 4: Proposed non-functional requirements

ID	Category	Requirement
NFR-01	Usability	A first-time student should be able to find an opportunity and open its details without instructions (to be checked in usability testing).
NFR-02	Responsiveness	Pages shall adapt to common mobile and desktop screen widths.
NFR-03	Security	Passwords shall be stored only as salted hashes using a recommended adaptive algorithm (e.g. BCrypt); never in plain text.
NFR-04	Security	Server-side role checks shall protect all administrator endpoints.
NFR-05	Security	All user-supplied input shall be validated on the server.
NFR-06	Security	Output rendered in the browser shall be encoded or sanitised to prevent script injection.

ID	Category	Requirement
NFR-07	Performance (provisional)	Listing and search pages should respond in about 2 seconds under typical demonstration load; to be reviewed after measurement.
NFR-08	Reliability	Database constraints (keys, uniqueness, not-null) shall protect data integrity.
NFR-09	Maintainability	Code shall follow a layered structure (controller, service, repository) with consistent naming.
NFR-10	Accessibility (goal)	Semantic HTML, keyboard operability, text alternatives and adequate contrast, aiming toward WCAG 2.2 AA where feasible.
NFR-11	Privacy	Only the minimum personal data (name, e-mail, password hash) shall be collected.
NFR-12	Security	Secrets and credentials shall be kept outside the repository (environment variables or untracked configuration).
NFR-13	Data quality	Official URLs shall be accepted only with http or https schemes and be well-formed.

6.3 Business rules

Table 5: Business rules

ID	Rule
BR-01	Only PUBLISHED opportunities are visible to visitors and students.
BR-02	Each opportunity belongs to exactly one category.
BR-03	A student can save a given opportunity only once.
BR-04	Each opportunity must carry an official source URL.
BR-05	Only administrators may create, change or delete content.
BR-06	ORVO is a reference tool; students must confirm details on the official site before applying.
BR-07	Content is entered manually; ORVO does not scrape other websites.

7 System Architecture and Design

ORVO is proposed as a three-tier web application: a browser-based frontend (HTML, CSS, JavaScript), a backend REST API (Java, Spring Boot) and a MySQL database. The backend is layered into a security layer, controllers, services and repositories. Official opportunity websites are reached only through links opened in the user’s browser.

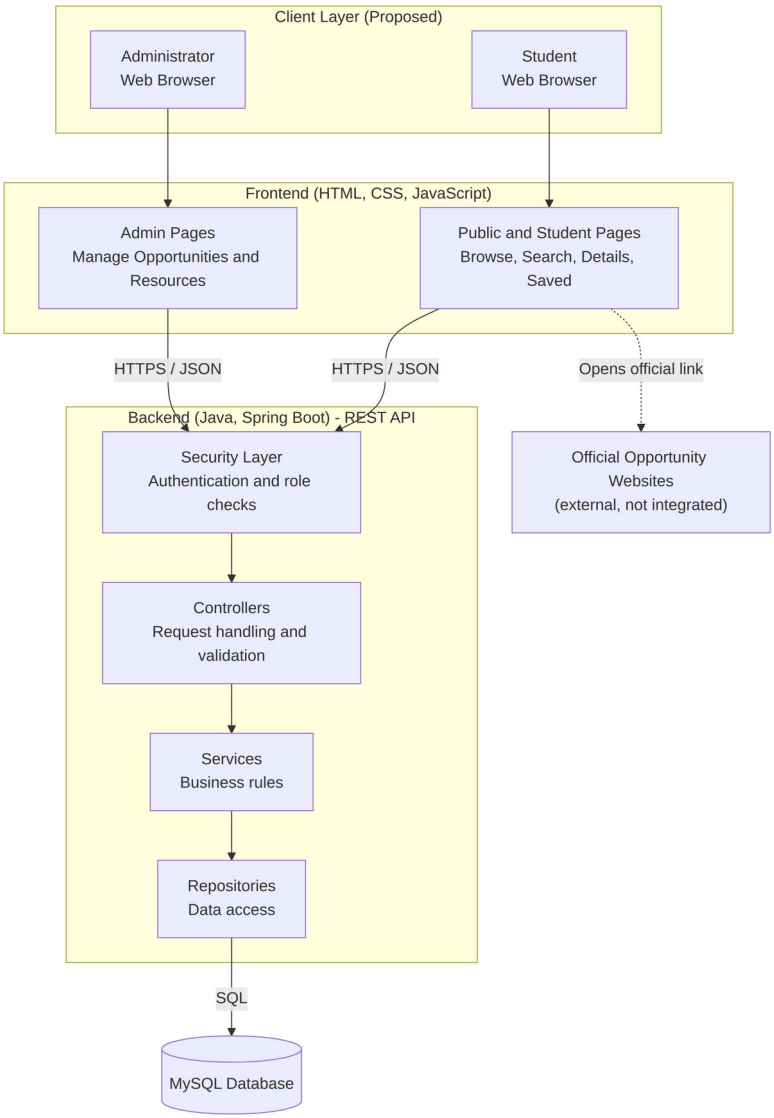


Figure 1: Proposed system architecture

Students and administrators use a browser. The frontend pages call the backend over HTTPS. The security layer checks who is calling and what they may do; controllers validate requests; services apply business rules; repositories read and write MySQL. The dotted line shows that ORVO only links to official sites.

Table 6: Component responsibilities

Component	Responsibility
Frontend pages	Render data, collect input, call API, basic client-side validation (convenience only)
Security layer	Authenticate users, enforce STUDENT/ADMIN roles
Controllers	Map HTTP requests to services, validate input, shape JSON responses
Services	Business rules (e.g. only published items visible, no duplicate saves, expiry marking)
Repositories	Database queries and persistence
MySQL	Durable storage with integrity constraints

Error handling. Invalid input returns 400 with field-level messages; unauthenticated access 401; forbidden access 403; missing items 404; duplicates 409; unexpected failures 500 with a generic message, with details kept in server logs only.

Scalability and maintainability. The MVP targets modest load. Layered code, a stateless API, indexed search columns and paginated lists allow later growth. Full-text search, caching or a frontend framework are possible later changes, not MVP commitments.

8 Workflow and Use Cases

The main journeys are: (1) discovering and filtering opportunities; (2) viewing details and linked resources; (3) saving opportunities and tracking deadlines and progress; and (4) administrator content management, where items are created as drafts, linked to resources, verified and then published.

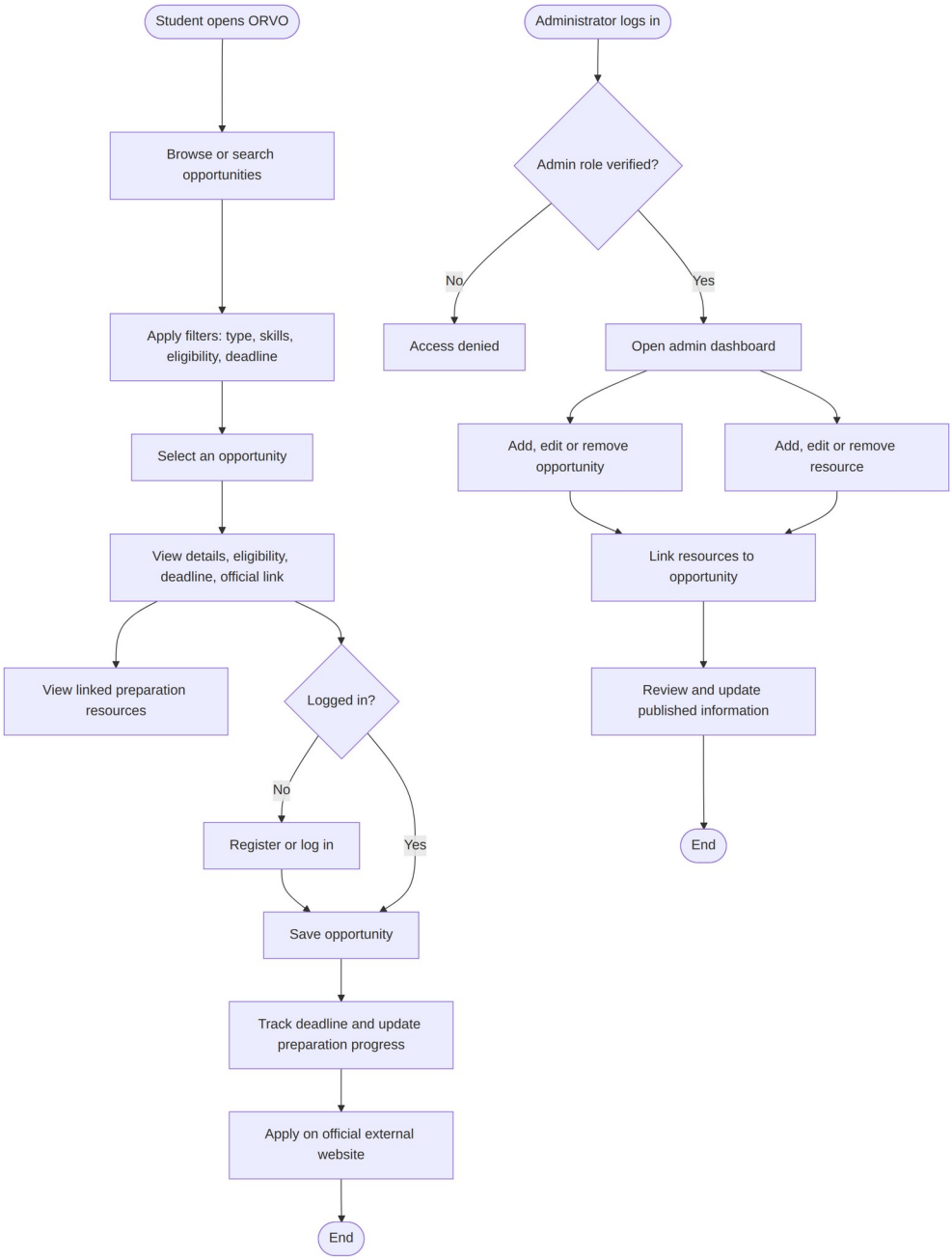


Figure 2: Student and administrator workflows

Figure 2 shows the student path from browsing to applying on the official website, and the administrator path from login with role verification to publishing verified content.

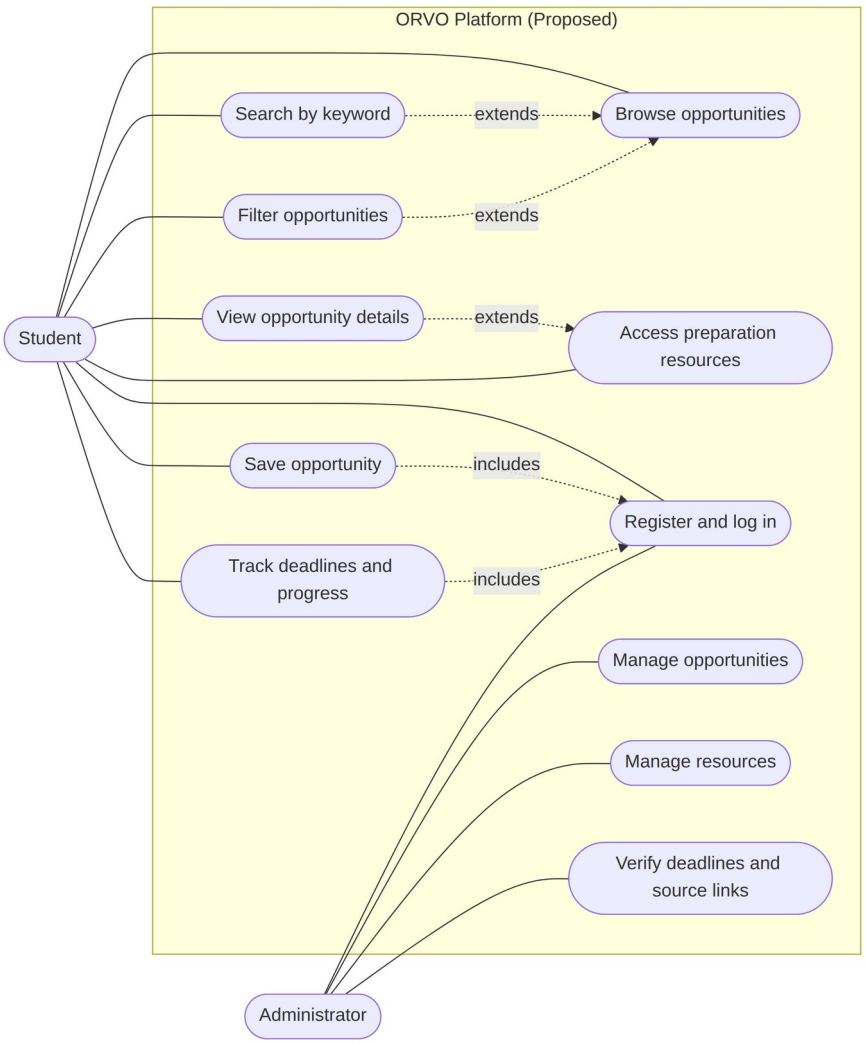


Figure 3: Use-case relationships (flowchart equivalent of a UML use-case diagram)

The two actors are the Student (a visitor before login) and the Administrator. Saving and tracking require login, which is shown as an “includes” relationship.

9 Database Design

A relational database is proposed because the data is structured and includes clear many-to-many relationships.

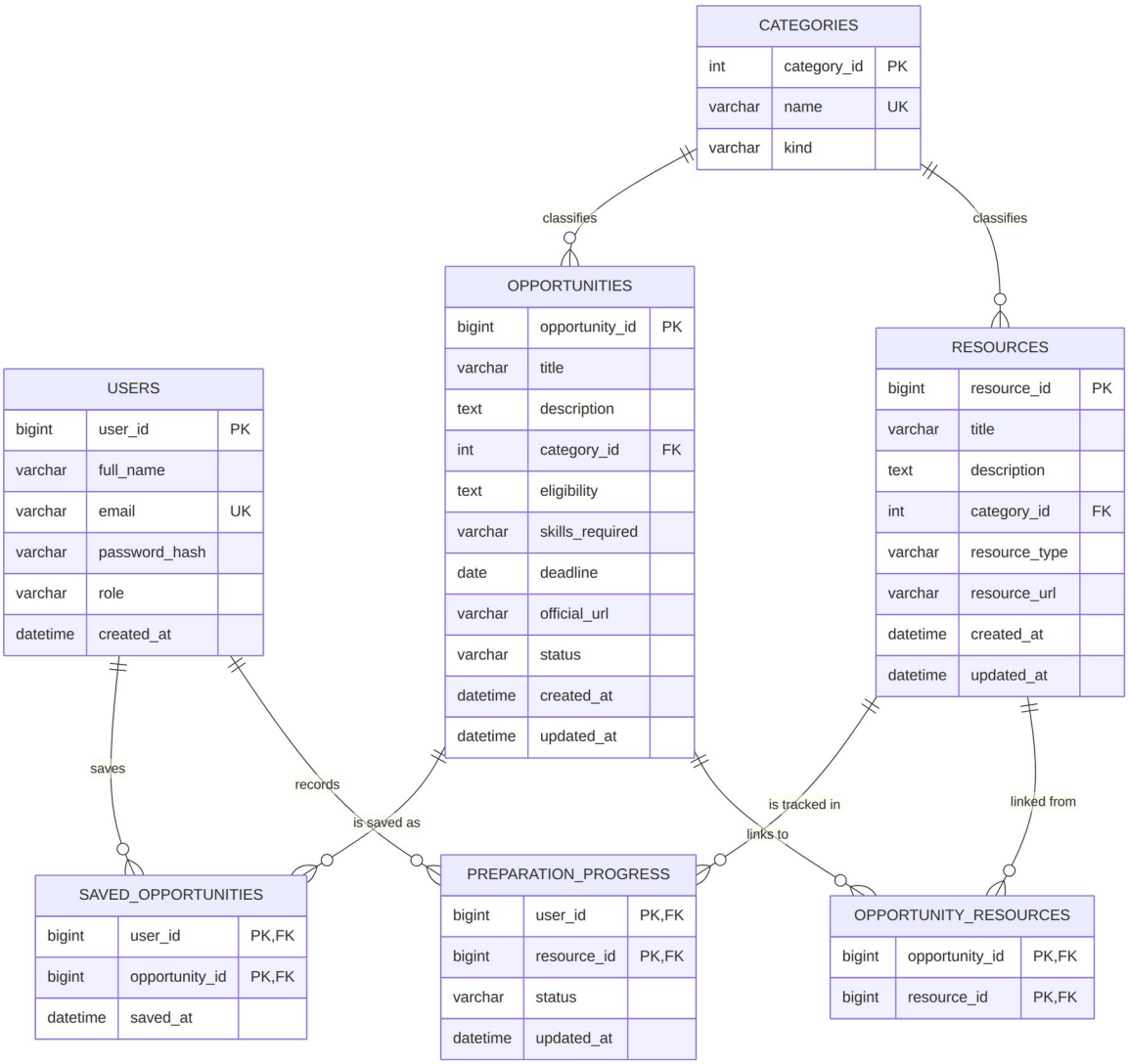


Figure 4: Proposed entity-relationship model

Key design decisions:

- Opportunity and Resource are related many-to-many (one guide may help several opportunities), through the link table OPPORTUNITY_RESOURCES.
- SAVED_OPPORTUNITIES uses a composite key of user and opportunity, enforcing “save once” at database level.
- PREPARATION_PROGRESS is tracked per user and resource, since preparation happens on resources; overall readiness for an opportunity can be derived from its linked resources.
- CATEGORIES is shared by opportunities and resources through a kind field, avoiding two near-identical tables.
- Allowed values: role = STUDENT or ADMIN; category kind = OPPORTUNITY or RESOURCE; opportunity status = DRAFT, PUBLISHED or ARCHIVED; resource type = TUTORIAL, PRACTICE, GUIDE or MATERIAL; progress status = NOT_STARTED, IN_PROGRESS or COMPLETED.
- Skills are stored as a text field in the MVP; a normalised skills table is a possible later improvement.
- No separate administrator table is needed: a role field on USERS is sufficient.

10 Technology Stack and Rationale

Table 7: Proposed technology stack ("Confirmed" means stated in the team's project context; "Recommended" means proposed for approval)

Layer	Technology	Status
Frontend	HTML, CSS, JavaScript	Confirmed (initial proposal)
Backend	Java, Spring Boot	Confirmed (initial proposal)
Database	MySQL	Confirmed (initial proposal)
Version control	Git, GitHub	Confirmed
Security library	Spring Security	Recommended
Build tool	Maven (or Gradle)	Recommended
Data access	Spring Data JPA	Recommended
Testing	JUnit 5, Spring Boot Test	Recommended
API testing	Postman or similar	Recommended (optional)
Hosting	Pending decision	Pending confirmation

Frontend (HTML, CSS, JavaScript) needs no build tooling, is easy to learn and debug, and suffices for listings, forms and filters; the trade-off is more manual DOM handling. **Java and Spring Boot** are mature, well documented and provide validation, security and testing support, at the cost of more configuration; Java and Spring Boot versions must be checked for compatibility before project creation. **MySQL** is free, widely used and fits structured, relational data. **Git and GitHub** provide version control, pull-request review and issue tracking. Alternatives such as React, Node.js or PostgreSQL were considered but not preferred, because the team context specifies the stack above. Deployment is undecided; local demonstration is the recommended baseline, with public hosting optional.

11 Security and Privacy Considerations

Table 8: Planned security and privacy controls

Area	Planned control
Passwords	Salted adaptive hashing (e.g. BCrypt); no plain-text storage; no password in logs
Authentication	E-mail and password; generic login error messages; limit repeated failed attempts (recommended)
Authorisation	Role checks (STUDENT, ADMIN) on every protected endpoint server-side; admin accounts created by team only
Input validation	Server-side validation of type, length, format for all fields; parameterised queries (via JPA) to prevent SQL injection
Output encoding	Encode/escape data before inserting into HTML (e.g. use textContent instead of innerHTML) to prevent XSS
Link validation	Official/resource URLs must be well-formed with http/https only; reject javascript: and similar schemes; open external links with rel="noopener noreferrer"
Secrets	Database credentials and keys in environment variables or untracked config; .gitignore for config files; never commit secrets
Transport	HTTPS wherever the application is publicly deployed
Data protection	Collect minimum data (name, e-mail, password hash); restrict database access to the backend account; backup approach pending
CSRF/session	Choose session or token approach; if cookies are used, enable CSRF protection and secure cookie flags
Dependencies	Keep libraries updated; review known-vulnerability reports for dependencies

Threats considered include injection, cross-site scripting, broken access control, credential theft, malicious links and stale or incorrect data. These correspond to categories in the OWASP Top 10. Only the minimum personal data (name, e-mail, password hash) would be collected.

12 Feasibility Analysis

Technical. The chosen technologies are widely documented, free and suitable for a CRUD-centred application; the MVP avoids complex integrations. **Operational.** Content is curated manually, avoiding dependence on external data feeds, but keeping content current requires continued effort. **Schedule.** Feasibility depends on team availability; a fixed MVP and phased plan reduce risk. **Economic.** Development uses free tools; hosting is undecided.

13 Proposed Test Plan

Testing is planned at unit, integration, system, usability and acceptance levels, using JUnit 5 and Spring Boot Test for automated tests and manual scripts for end-to-end and usability checks. Risk-based priorities are: (1) access control and authentication, (2) correctness of deadlines and links, (3) search and filter, (4) data integrity, (5) usability and accessibility. Defects are tracked as GitHub Issues with severity, owner, fix branch and retest before closure. Full case details (preconditions and procedures) are in the repository document *05 Security and Test Plan*.

All test cases are NOT RUN, because no implementation exists.

Table 9: Proposed test cases (all NOT RUN)

ID	Objective	Expected result	Status
TC-01	List published opportunities (FR-01)	Only published items shown	NOT RUN
TC-02	Keyword search (FR-02)	Matching items only; empty result handled	NOT RUN
TC-03	Filter (FR-03)	Results match all selected filters	NOT RUN
TC-04	View details (FR-04)	Description, eligibility, deadline, official link shown correctly	NOT RUN
TC-05	Linked resources (FR-05)	Exactly those 2 resources shown	NOT RUN
TC-06	Resource list/search (FR-06)	Correct matches shown	NOT RUN
TC-07	Register, login, logout (FR-07)	Success flows work; wrong password rejected with generic message	NOT RUN
TC-08	Save/unsave (FR-08)	Saved once; duplicate prevented; unsave removes	NOT RUN
TC-09	Saved list by deadline (FR-09)	Sorted by deadline; passed deadlines marked	NOT RUN
TC-10	Progress status (FR-10)	Status persists	NOT RUN
TC-11	Admin creates opportunity (FR-11)	Valid saved as draft; invalid rejected with messages	NOT RUN
TC-12	Admin edit/archive/delete (FR-12)	Changes applied; archived hidden from students	NOT RUN
TC-13	Admin manages resources/categories (FR-13, FR-17)	Changes persist; deleting an in-use category is handled safely	NOT RUN
TC-14	Link/unlink resources (FR-14)	Detail page reflects each change	NOT RUN
TC-15	URL validation (NFR-13, BR-04)	Unsafe/malformed rejected; valid accepted	NOT RUN
TC-16	Password storage (NFR-03)	Stored value is a salted hash, not the password	NOT RUN
TC-17	Expiry marking (FR-16)	Marked as expired	NOT RUN
TC-18	Publication status (FR-15)	Hidden while draft; visible after publishing	NOT RUN
TC-19	Admin access control (NFR-04)	401/403 returned; no data changed	NOT RUN

ID	Objective	Expected result	Status
TC-20	Input validation / injection (NFR-05)	No error leak, no data exposure, input rejected or safely handled	NOT RUN
TC-21	Output encoding (NFR-06)	Text displayed literally; no script runs	NOT RUN
TC-22	Usability (NFR-01)	Tasks completed; issues recorded	NOT RUN
TC-23	Responsive/accessibility check (NFR-02, NFR-10)	Layout adapts; core flows keyboard-usable; issues recorded	NOT RUN
TC-24	Secret scan (NFR-12)	None found	NOT RUN
TC-25	Provisional response time (NFR-07)	Compare with provisional 2 s target; record result	NOT RUN

14 Development Plan and Milestones

The documentation deadline (two days) is separate from the software development schedule, which is tentative, assumes about eight weeks and must be adjusted to the college's actual deadline (pending confirmation).

Table 10: Development phases and deliverables

Phase	Goal	Deliverables
0	Documentation	This documentation set, PDF/DOCX report
1	Foundation	Spring Boot project, MySQL schema, sample data, frontend skeleton
2	Core features	Opportunity and resource listing, search, filter, details, linking
3	User features	Registration/login, saved opportunities, deadline view, progress
4	Administration and security	Admin pages, role checks, validation, hardening
5	Quality and delivery	Test execution and report, bug fixes, final documentation, demo

Table 11: Tentative two-day documentation schedule

When	Tasks
Day 1 morning	Team reviews concept, scope and MVP; confirm institutional details and repository URL; approve or change proposals
Day 1 afternoon	Review Markdown documents and diagrams for accuracy; fix terminology
Day 2 morning	Review PDF/DOCX report; fill placeholders (college, department, course, year)
Day 2 afternoon	Upload to GitHub, verify links render, final read-through, submit

Table 12: Tentative software development milestones

Weeks	Phase	Milestone
1	1	M1: Project runs locally; schema created
2–3	2	M2: Browse/search/filter/details working with sample data
4–5	3	M3: Accounts, saving, progress working
6	4	M4: Admin interface and access control working
7	5	M5: Tests executed, defects triaged
8	5	M6: Fixes, final documentation, demonstration

A milestone is reached only when its features are actually implemented and tested. Collaboration uses a protected main branch, short-lived feature, fix and docs branches, and pull requests reviewed by a member other than the author.

15 Proposed Team Responsibilities

These are suggestions for discussion only; no work has been assigned or completed. There is no designated team leader, so decisions are made by agreement with a rotating weekly facilitator. Every member reviews others' pull requests.

Table 13: Proposed team responsibilities (subject to approval)

Member	Proposed primary area	Secondary
Dheeraj Mandal	Backend API and security	Code review
Jatin Lawaniya	Database schema and sample data	Backend services
Aakash Pal	Frontend student pages	UI consistency
Abhishek Tyagi	Admin pages and testing coordination	Documentation upkeep

16 Expected Outcomes

If the proposal is approved and implemented: a working MVP web application; a documented REST API; a normalised MySQL schema; a test suite and a test report produced from real test execution; and updated documentation. None of these exist yet.

17 Risks, Assumptions and Constraints

Table 14: Risk register (ratings are team judgements, not measured values)

ID	Risk	Likelihood	Impact	Mitigation
R1	Scope grows beyond MVP	High	High	Fixed MVP list; defer extras to “Later”
R2	Team members unfamiliar with Spring Boot	Medium	High	Early spike, pair work, official guides
R3	Outdated or wrong opportunity data	High	Medium	Admin verification, expiry marking, BR-06 notice
R4	Integration problems between frontend and API	Medium	Medium	Agree API contract early; test with Postman
R5	Merge conflicts, inconsistent code	Medium	Medium	Small PRs, branch rules, shared conventions
R6	Security flaws (access control, XSS)	Medium	High	Planned tests TC-16, TC-19–TC-21, reviews
R7	Too little time for testing	Medium	High	Write test cases early; reserve Week 7
R8	Unavailable team member	Medium	Medium	Shared ownership, documented work
R9	Hosting unavailable	Medium	Low	Local demonstration as fallback

Assumptions (pending approval): at least one member acts as administrator; data is entered manually; the team is familiar with the stack or will learn it; official links remain available. **Constraints:** the two-day documentation deadline, student-team capacity, a fixed technology stack, and no prescribed college format.

18 Future Scope

After the MVP, and only if justified: notifications for approaching deadlines, AI-based personalised opportunity recommendations, hackathon team matching, personalised preparation plans and further career-guidance features. These are excluded from the initial scope.

19 Conclusion

ORVO targets a specific, practical gap: the separation of opportunity discovery from preparation. A deliberately limited MVP built on a familiar technology stack gives the team a realistic route from this proposal to a working system. The project is currently at the proposal stage and all functionality described is planned, not implemented.

References

The sources below are official or primary. They have not yet been opened and verified by the team; access dates are to be completed on verification. Entries 14 to 17 are cited only as examples of platform types.

Table 15: References

#	Title	Publisher	URL
1	Spring Boot (project page and reference documentation)	VMware / Spring	https://spring.io/projects/spring-boot
2	Spring Security Reference	VMware / Spring	https://docs.spring.io/spring-security/reference/

#	Title	Publisher	URL
3	MySQL Reference Manual	Oracle	https://dev.mysql.com/doc/
4	Java Platform documentation	Oracle	https://docs.oracle.com/en/java/
5	HTML, CSS and JavaScript documentation (MDN Web Docs)	Mozilla	https://developer.mozilla.org/
6	Git documentation	Git project	https://git-scm.com/doc
7	GitHub Docs	GitHub	https://docs.github.com/
8	OWASP Top 10	OWASP Foundation	https://owasp.org/www-project-top-ten/
9	Password Storage Cheat Sheet	OWASP Foundation	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
10	Web Content Accessibility Guidelines (WCAG) 2.2	W3C	https://www.w3.org/TR/WCAG22/
11	ISO/IEC/IEEE 29148:2018, Requirements engineering	ISO	https://www.iso.org/standard/72089.html
12	JUnit 5 User Guide	JUnit team	https://junit.org/junit5/docs/current/user-guide/
13	Mermaid documentation	Mermaid project	https://mermaid.js.org/
14	Devpost (example hackathon platform)	Devpost	https://devpost.com/
15	Internshala (example internship platform)	Internshala	https://internshala.com/
16	LeetCode (example practice platform)	LeetCode	https://leetcode.com/
17	Coursera (example learning platform)	Coursera	https://www.coursera.org/

Appendix A: Illustrative API Endpoints

Paths are illustrative and may change during implementation.

Table 16: Proposed API responsibilities

Method and path	Purpose	Access
GET /api/opportunities?q=&category=&skill=&deadlineBefore=	List/search/filter published opportunities	Public
GET /api/opportunities/{id}	Details with linked resources	Public
GET /api/resources?q=&type=	List/search resources	Public
POST /api/auth/register, POST /api/auth/login	Account creation, login	Public
GET/POST/DELETE /api/me/saved	List, save, unsave opportunities	Student
PUT /api/me/progress/{resourceId}	Set preparation status	Student
POST/PUT/DELETE /api/admin/opportunities[/{id}]	Manage opportunities	Admin
POST/PUT/DELETE /api/admin/resources[/{id}]	Manage resources	Admin
PUT/DELETE /api/admin/opportunities/{id}/resources/{resourceId}	Link/unlink	Admin

Appendix B: Unresolved Decisions and Placeholders

Table 17: Unresolved items

Item	Status
College, department, course, academic year	Pending confirmation
Repository URL	Pending confirmation
Approval of scope, requirements and technology stack	Pending team approval
Team responsibilities	Pending team approval
Java, Spring Boot and MySQL versions	Pending confirmation
Session or token authentication	Pending decision
Hosting approach	Pending decision
Licence	Not selected
Final software development deadline	Pending confirmation